

Recommender sistemi u eKnjiga

Ovaj dokument opisuje hibridni sistem preporuke implementiran u `RecommendationService`.

Sistem kombinuje Collaborative Filtering (CF KNN) i heuristički pristup.

Primarno koristi ocjene korisnika, a u slučaju nedostatka podataka koristi kupovine, kategorije, autore i rating knjiga.

1) Glavni recommender (`RecommendationService`)

Tok rada

1. Poziva se `GetRecommendedAsync`
2. Pokreće se Collaborative Filtering (CF KNN)
3. Ako CF vrati rezultate, oni se vraćaju korisniku
4. Ako CF ne vrati rezultate, koristi se heuristički recommender
5. Rezultati se mapiraju u `BookResponse` i vraćaju API-ju

2) Collaborative Filtering (`GetRecommendedCfKnnAsync`)

Implementira user-based KNN Collaborative Filtering koristeći cosine similarity između korisnika.

Tok rada

1. Učitava ocjene target korisnika
2. Ako korisnik nema ocjena, prekida se CF
3. Pronalazi druge korisnike koji su ocjenjivali iste knjige (overlap)
4. Računa cosine similarity između target korisnika i ostalih
5. Uzima top K najsličnijih korisnika (default: 25)
6. Pronalazi knjige koje su ocijenili susjedi, ali target korisnik nije
7. Računa predikciju ocjene
8. Sortira i vraća top N knjiga

Ključni dijelovi koda

Učitavanje ocjena korisnika

```
var targetRatingsList = await _ctx.Reviews
    .Where(r => r.UserId == userId)
    .Select(r => new { r.BookId, r.Rating })
    .ToListAsync();
```

Cosine similarity

```
var denom = normU * Math.Sqrt(normV2[v]);
var sim = denom == 0 ? 0 : kv.Value / denom;
```

Matematički izraz:

$$sim(u, v) = \frac{\sum_{i=1}^n (r_{ui} \cdot r_{vi})}{(\|u\| \cdot \|v\|)}$$

Predikcija ocjene

```
num[r.BookId] += sim * r.Rating;
den[r.BookId] += Math.Abs(sim);
```

Formula:

$$predictedScore = \frac{\sum sim(u,v) \cdot rating}{\sum (|sim(u,v)|)}$$

Filtriranje kandidata

Knjiga mora:

- biti ocijenjena od susjeda
- ne biti ocijenjena od target korisnika
- ne biti kupljena od target korisnika
- (opcionalno) pripadati traženoj kategoriji, ako se proslijedi kategorija

3) Heuristički recommender (GetRecommendedHeuristicAsync)

Koristi se kada CF nema dovoljno podataka.

Tok rada

1. Učitava knjige koje je korisnik kupio
2. Ako korisnik nema kupovina, aktivira se cold-start
3. Ako ima kupovine, gradi se korisnički profil
4. Generišu se kandidati
5. Skoriraju se knjige
6. Sortiraju se i vraća se top N

Cold-start logika

Ako korisnik nema kupovina:

```
.OrderByDescending(b => (b.Rating * (b.RatingCount + 1)))  
.ThenByDescending(b => b.CreatedAt)
```

Sistem vraća:

- najbolje ocijenjene knjige
- novije knjige imaju prednost

Izgradnja korisničkog profila

Iz kupljenih knjiga izvlače se:

- kategorije
- autori

Broji se koliko puta se pojavljuju:

```
.GroupBy(x => x)  
.ToDictionary(g => g.Key, g => g.Count());
```

Rezultat:

- favCats
- favAuthors

Skoriranje kandidata

Za svaku knjigu:

1. Dodaje se težina za podudaranje kategorije.
2. Dodaje se težina za podudaranje autora.
3. Dodaje se bonus na osnovu ratinga:

score += Rating × 10

Završno sortiranje

```
.OrderByDescending(x => x.Score)  
.ThenByDescending(x => x.Book.CreatedAt)
```

4) Personalizirane slične knjige (GetPersonalizedSimilarAsync)

Preporučuje knjige slične konkretnoj knjizi, uz personalizaciju.

Tok rada

1. Učitava ciljanu knjigu.
2. Izvlači njene kategorije i autore.
3. Učitava korisnikove kupovine.
4. Gradi korisnički profil.
5. Generiše kandidate.
6. Računa score.
7. Sortira i vraća rezultate.

Skoriranje

Ako kandidat:

- dijeli kategoriju onda +2 boda
- dijeli autora onda +3 boda
- odgovara korisničkim preferencama onda se dodaje frekvencijska težina
- ima visok rating onda se dodaje $\text{Rating} \times 10$

5) Mapiranje rezultata

Rezultati se mapiraju u **BookResponse** koji sadrži:

- osnovne podatke o knjizi
- autore
- kategorije
- rating i broj ocjena
- datum kreiranja

```
private static BookResponse MapBookToResponse(Book b)
```

6) Sažetak

Implementirani sistem preporuke u okviru **RecommendationService** zasniva se na hibridnom pristupu koji kombinuje Collaborative Filtering i heurističko bodovanje.

Primarni mehanizam koristi user-based Collaborative Filtering sa cosine similarity mjerom sličnosti između korisnika, te weighted prediction formulom za procjenu relevantnosti knjiga. Ovaj pristup omogućava personalizirane i precizne preporuke za korisnike koji imaju istoriju ocjenjivanja.

U situacijama kada nema dovoljno podataka za Collaborative Filtering (npr. novi korisnik bez ocjena), aktivira se heuristički sistem koji koristi informacije o kupovinama, kategorijama, autorima i globalnom ratingu knjiga. Time se osigurava stabilno ponašanje sistema i rješavanje cold-start problema.

Dodatno, implementirana je funkcionalnost personaliziranih sličnih knjiga, koja kombinuje sličnost prema ciljanoj knjizi i individualne preferencije korisnika.

Filtriranje kandidata vrši se na nivou baze podataka, čime se optimizuju performanse i smanjuje nepotrebna obrada podataka u memoriji.

Kombinacijom ovih pristupa sistem postiže balans između preciznosti, personalizacije i skalabilnosti, te predstavlja robusno rješenje za preporuku knjiga u aplikaciji eKnjiga.